OFFLINE PASSWORD MANAGER

A PROJECT REPORT SUBMITTED IN FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

By

Batch – 9

K. Roopadevi (323103210099)

K**. Thanmaya** (323103210098)

G. Mohana Priya (323103210242)

K. **Sudha Rani** (323103210244)

Under the esteemed guidance of

Dr. N. Sharmili

Professor

CSE Department

Department of Computer Science and Engineering

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

[Approved by AICTE NEW DELHI, Affiliated to Andhra University, Visakhapatnam]

[Accredited by National Board of Accreditation (NBA) for B. Tech. CSE, ECE & IT – valid from 2019-22 and 2022-25]

[Accredited by National Assessment and Accreditation Council (NAAC) with A Grade-Valid from 2022-2027]

Kommadi, Madhurawada, Visakhapatnam – 530048

2023 – 2027

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report titled "**Offline Password Manager "** is the Bonafide work of the following III B. Tech students from the Department of Computer Science and Engineering, Gayatri Vidya Parishad College of Engineering for Women, affiliated with Andhra University, Visakhapatnam. This project was done as **part of Cryptography and Network Security lab** during the academic year 2025-2026 (III Year Semester-2).

K. Roopadevi (323103210099)

K**. Thanmaya** (323103210098)

G. Mohana Priya (323103210242)

K. **Sudha Rani** (323103210244)

Signature of the Guide

Dr. N. Sharmili MTech., Ph.D.

Professor,

Dept. of CSE

Signature of the HOD

Dr. P. V. S. L. Jagadamba MTech, Ph.D.

Professor,

Dept. of CSE

External Examiner

ABSTRACT

Offline Password Manager with Honey Password Protection is a secure local credential vault designed to help users register, log in, store service passwords, and retrieve them safely without depending on cloud services. The system uses a master password for authentication and protects stored data with local encryption in a SQLite database. A key feature of the project is honey password behaviour. When a user enters the correct master password, the application returns the real service password. When a similar or deceptive password is entered, the system returns a honey password instead of the real one. After a honey-mode interaction, the correct master password still reveals the actual service password, preserving both security and usability.

The project is implemented using Python, Flask, SQLite, HTML, CSS, and Jinja2. It also uses Argon2 for password hashing, key derivation for vault encryption, and HMAC validation for integrity checking. This makes the application lightweight, offline-capable, and suitable for secure personal password management.

1. SYSTEM REQUIREMENTS

1.1 Software Requirements

Software	Version / Specification
Python	3.x
Flask	Web framework
SQLite	Local database
HTML	Frontend structure
CSS	Frontend styling
Jinja2	Template rendering
Visual Studio Code	Code editor
Operating System	Windows / Linux / macOS

Table 1. Software Requirements

1.2 Python packages

S.No.	Package Used	Description
1	Flask	Backend web framework
2	argon2-cffi	Master password hashing
3	sqlite3	Database management
4	hashlib	Key derivation and hashing

5	hmac	Integrity protection
6	json	Honey metadata storage
7	secrets	Secure token generation
8	pathlib	File path handling

Table 2. Python Packages

1.3 Front End Packages used

S.No.	Libraries Used	Description
1	HTML	Builds the web pages
2	CSS	Styles the interface
3	Jinja2	Connects frontend with Flask
4	Browser	Runs the local web application

Table 3. Front end Packages

1.4 Hardware Requirements

The below Table 4 shows the minimum hardware requirements of our project.

S.No.	Hardware Component	Specification
1	Processor	Intel i3 or above

2	RAM	4 GB minimum
3	Storage	500 MB free space
4	Display	Monitor / laptop screen
5	Input	Keyboard and mouse
6	Internet	Not required for normal use

Table 4. Minimum Hardware Requirements

2. METHODOLOGY

2.1 System Architecture / Work Flow Diagram

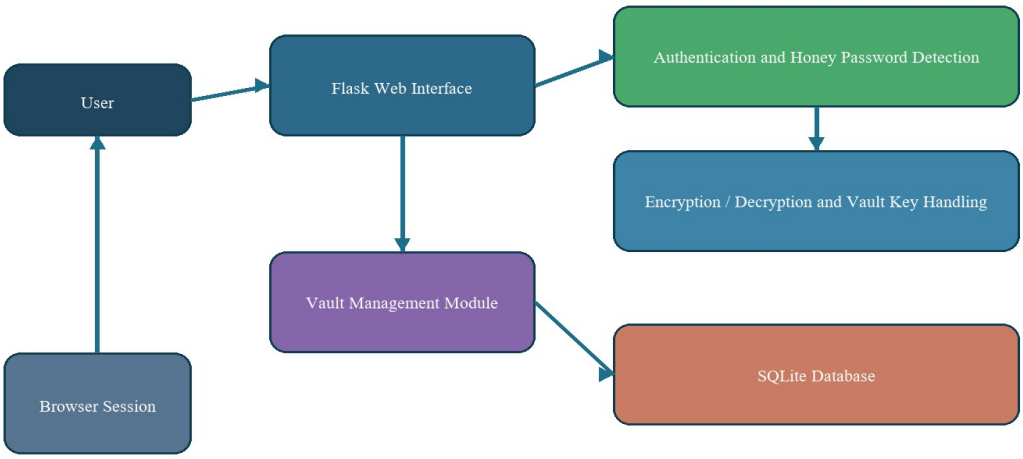


Fig 2.1: Offline Password Manager Architecture

The system architecture consists of multiple modules working together to provide secure local password management. The workflow begins with user registration and master password setup, followed by secure login, vault access, password saving, and conditional password retrieval based on whether the provided password is correct or similar to the original master password.

2.2 Key Components and Workflow

Phase 1: Registration and Master Password Setup

The registration module collects a username and master password, validates it, hashes it with Argon2, and generates both real-vault and honey-vault credentials.

Phase 2: Login and Vault Opening

When the correct master password is entered, the real vault opens. Similar or honey passwords are recognized so the system can serve decoy behaviour without revealing real secrets.

Phase 3: Password Save and Retrieval

Users can save service passwords such as Instagram or Gmail. Retrieval depends on the authentication password entered at access time. Correct master password returns the actual password, while a similar password returns the honey password.

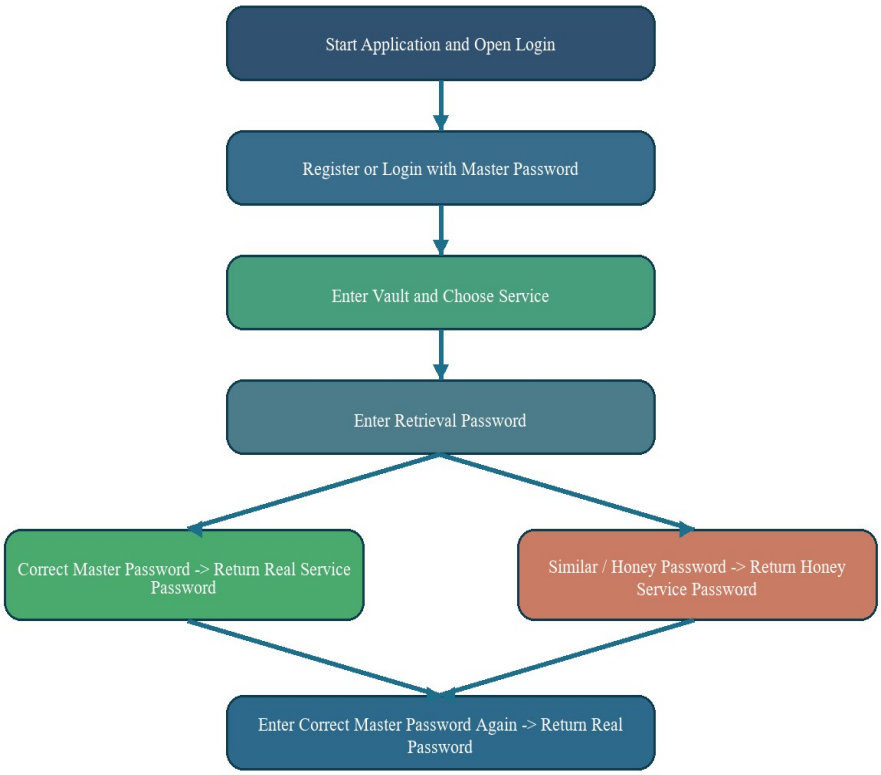


fig 2.2: workflow for correct and honey password retrieval

2.3 Algorithm

Offline Password Manager

Registration Phase

1. Start
2. User opens application

3. Enter username and master password
4. Hash the master password using secure hashing algorithm
5. Generate salt for additional security
6. Store username, hashed password, and salt in database
7. Initialize empty password vault
8. End Registration

Login Phase

9. User enters username and master password
10. Retrieve stored hash and salt
11. Hash entered password with salt
12. Compare with stored hash

Decision Phase

13. If password is correct:
    - Grant access to vault
14. If password is incorrect:
    - Trigger honey password mechanism
    - Generate fake (honey) vault data
    - Provide limited or deceptive access
15. Log login attempt

Vault Operations Phase

16. User selects operation: Save / Retrieve password

Save Password

17. Enter service name and password
18. Encrypt password using encryption algorithm
19. Store encrypted password in database
20. Confirm save operation

Retrieve Password

21. Enter service name
22. Fetch encrypted password
23. Decrypt password

Final Decision

24. If login was valid:

- Show real password

25. If honey mode active:
    - Show fake password
26. Log operation details
27. Display result
28. End

2.4 Modules and Descriptions

2.4.1 User Authentication Module Handles user login and registration.

Functions:
- User registration
- Login authentication
- Password hashing and verification

2.4.2 Vault Management Module Manages storage and retrieval of passwords.

Functions:
- Save passwords
- Retrieve passwords
- Manage user vault

2.4.3 Cryptography Module Ensures data security.

Functions:
- Encrypt passwords
- Decrypt passwords
- Secure key handling

2.4.4 Honey Password Module Provides deception-based security.

Functions:
- Generate honey passwords

• Detect incorrect login attempts
• Return fake data for attackers

2.4.5 Database Module Handles

data storage.

Functions:
• Store user credentials
• Store encrypted passwords
• Retrieve stored data

2.4.6 Interface Module Handles

frontend interaction.

Functions:
• Display login/register pages
• Show vault interface
• Handle user inputs

2.4.7 Log Module

Maintains system activity logs.

Functions:
• Record login attempts
• Track save/retrieve actions
• Maintain timestamps

3. IMPLEMENTATION

The **Offline Password Manager** is implemented using Python. It securely stores user passwords in a local database and allows access using a master password.

The system has the following main parts:

- **User Login:**
  The user enters a master password. It is verified using hashing to ensure security.
- **Password Storage:**
  Passwords are stored in encrypted form so that no one can read them directly.
- **Password Management:**
  The user can:

- ➢ Add new passwords
- ➢ View saved passwords ➢ Update or delete passwords

- **Database:** ll data is stored locally using SQLite (offline storage).
- **Encryption:**
  Passwords are encrypted before saving and decrypted only when needed.

Testing

The system was tested with different scenarios:

- Correct and incorrect master password
- Adding new credentials
- Retrieving stored passwords
- Updating and deleting entries
- Handling empty or invalid inputs

File Structure

```
offline_password_manager/
│
├── data/
│   └── (likely contains DB or keys)
│
├── static/
│   └── styles.css
│
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   └── vault.html
│
├── app.py
├── crypto_utils.py
├── database.py
├── main.py
├── security.py
├── vault_core.py
│
├── vault.db
│
```

```
├── README.md
├── requirements.txt
```

SOURCE CODE:

Code of Flask Application (app.py) from __future__ import annotations import sqlite3

from functools import wraps from pathlib import path import secrets from flask import

flask, abort, flash, g, redirect, render_template, request, session, url_for from database

import (create_user, fetch_service_entry,

  get_db,

get_user_by_username,

init_db, list_service_names,

save_service_secret,

)

from security import (

decrypt_secret, derive_master_key,

generate_honey_secret,

unlock_vault_key,

verify_password_or_honey,

)

base_dir = path(__file__).resolve().parent

secret_file = base_dir / "data" / ".flask_secret_key"

active_vaults: dict[str, dict[str, str | int]] = {} def

_load_or_create_secret_key() -> str:

  secret_file.parent.mkdir(parents=true, exist_ok=true)

if secret_file.exists():

    return secret_file.read_text(encoding="utf-8").strip()

secret_key = secrets.token_urlsafe(48)

secret_file.write_text(secret_key, encoding="utf-8")

return secret_key def issue_csrf_token() -> str:

```
    token = session.get("csrf_token")
if not token:
        token = secrets.token_urlsafe(24)
    session["csrf_token"] = token

    return token def
validate_csrf() -> none:
    submitted = request.form.get("csrf_token", "")     stored =
session.get("csrf_token", "")   if not submitted or not stored or not
secrets.compare_digest(submitted, stored):
        abort(400) def
create_active_vault(
user_id: int,
username: str,     mode:
str,     vault_key: str,
fake_vault_key: str,
) -> str:
    vault_token = secrets.token_urlsafe(32)
active_vaults[vault_token] = {
"user_id": user_id,
        "username": username,
        "vault_mode": mode,
        "vault_key": vault_key,
        "fake_vault_key": fake_vault_key,
    }
    return vault_token def
clear_active_vault() -> none:
    vault_token = session.get("vault_token")
if vault_token:
```

```
        active_vaults.pop(vault_token, none)
def create_app() -> flask: app =
flask(__name__)
app.config["secret_key"] =
_load_or_create_secret_key()
app.config["database"] = base_dir /
"data" / "vault.db"
app.config["session_cookie_httponly"] =
true
app.config["session_cookie_samesite"] =
"lax"
app.config["session_cookie_name"] =
"offline_vault_session"
app.config["permanent_session_lifetime"
] = 1800
init_db(app.config["database"])
@app.before_request     def
before_request() -> none:
        g.db = get_db(app.config["database"])
        g.csrf_token = issue_csrf_token()
        g.active_vault = none         vault_token
= session.get("vault_token")         if
vault_token:
            g.active_vault = active_vaults.get(vault_token)
@app.teardown_request     def teardown_request(_exception:
baseexception | none) -> none:
        db = g.pop("db", none)
if db is not none:
            db.close()
```

@app.errorhandler(400) def
handle_bad_request(_error):
flash("the request could not be validated. please try again.", "error")
if session.get("user_id"):
return redirect(url_for("vault"))
return redirect(url_for("login"))
@app.errorhandler(500) def
handle_server_error(_error):
flash("something went wrong
internally. please retry your
action.", "error") if
session.get("user_id"):
return redirect(url_for("vault")) return
redirect(url_for("index")) return app app = create_app() def
login_required(view): @wraps(view) def wrapped_view(*args,
**kwargs): if not session.get("user_id") or not getattr(g,
"active_vault", none):
clear_active_vault()
session.clear() flash("please log in to
continue.", "error") return
redirect(url_for("login")) return view(*args,
**kwargs) return wrapped_view
@app.route("/") def index(): if
session.get("user_id"):
return redirect(url_for("vault")) return
render_template("index.html")
@app.route("/register", methods=["get", "post"])
def register(): if request.method == "post":

```
        validate_csrf()         username = request.form.get("username",
"").strip()         password = request.form.get("password", "")
confirm_password = request.form.get("confirm_password", "")
errors: dict[str, str] = {} if not username:
            errors["username"] = "* required"         elif
len(username) < 3   errors["username"] = "use at least 3
characters"         elif not username.replace("_", "").replace("-",
"").isalnum():             errors["username"] = "use letters,
numbers, - or _ only"         if not password:
            errors["password"] = "* required"
if not confirm_password:
            errors["confirm_password"] = "* required"
elif password != confirm_password:
            errors["confirm_password"] = "passwords do not match"
if not errors and len(password) < 12:
            errors["password"] = "use at least 12 characters"
elif not errors and password.lower() == password:
            errors["password"] = "include at least one uppercase letter"
elif not errors and not any(char.isdigit() for char in password):
            errors["password"] = "include at least one number"         elif not
errors and not any(not char.isalnum() for char in password):
            errors["password"] = "include at least one symbol"
        if errors:
            return render_template("register.html", errors=errors, form=request.form)
        try:
            create_user(g.db, username=username, password=password)
except sqlite3.integrityerror:
            errors["username"] = "username already exists"         return
    render_template("register.html", errors=errors, form=request.form)
```

```
    flash("registration complete. use your master password to open the real
    vault.", "success") return redirect(url_for("login")) return
    render_template("register.html", errors={}, form={})


@app.route("/login", methods=["get", "post"])
def login():     if request.method == "post":
        validate_csrf()         username =
request.form.get("username", "").strip()         password =
request.form.get("password", "")     errors: dict[str, str] =
{}         if not username:
            errors["username"] = "* required"
if not password:
            errors["password"] = "* required"
        if errors:
            return render_template("login.html", errors=errors, form=request.form)
user = get_user_by_username(g.db, username)
        if not user:
            errors["username"] = "unknown username"            return
render_template("login.html", errors=errors, form=request.form)
        resolved_login = verify_password_or_honey(password, user["password_hash"],
user["honey_password_hashes"])
        if not resolved_login:
            errors["password"] = "invalid password"            return
render_template("login.html", errors=errors, form=request.form)         mode,
honey_index = resolved_login         clear_active_vault()         session.clear()

        vault_key = unlock_vault_key(
password,
user["encryption_salt"],
mode,            user["real_key_wrap"],
```

```
user["honey_key_wraps"],
honey_index,
        )
        fake_vault_key = (              decrypt_secret(
derive_master_key(password, user["encryption_salt"]),
user["fake_key_wrap_for_real"],
            )
            if mode == "real"
else vault_key
        )
        vault_token = create_active_vault(
user_id=user["id"],
username=user["username"],
mode=mode,              vault_key=vault_key,
fake_vault_key=fake_vault_key,
        )
        session["user_id"] = user["id"]
session["username"] = user["username"]
session["vault_token"] = vault_token
session.permanent = true
        flash(
            "vault unlocked successfully.",
            "success",
        )
        return redirect(url_for("vault"))     return
render_template("login.html", errors={}, form={})
@app.route("/vault", methods=["get", "post"])
@login_required
```

```
def vault():     errors: dict[str,
str] = {}     result = none
active_vault = g.active_vault
if request.method == "post":
        validate_csrf()         action = request.form.get("action")
service_name = request.form.get("service_name", "").strip()
service_password = request.form.get("service_password", "")
auth_password = request.form.get("auth_password", "")         if
not service_name:
            errors["service_name"] = "* required"
if action == "save":             if not
service_password:
                errors["service_password"] = "* required"
if not errors:                 save_service_secret(
                    g.db,
                    user_id=active_vault["user_id"],
    service_name=service_name,
    actual_secret=service_password,
    honey_secret=generate_honey_secret(service_password),

                    real_vault_key_hex=active_vault["vault_key"] if active_vault["vault_mode"] ==
"real" else none,
fake_vault_key_hex=active_vault["fake_vault_key"],
mode=active_vault["vault_mode"],
                )
                flash("service password saved securely in the local database.", "success")
return redirect(url_for("vault"))         if action == "get":             if not
auth_password:
```

```
            errors["auth_password"] = "* required"    user =
get_user_by_username(g.db, session["username"])
resolved_login = verify_password_or_honey(
auth_password,                user["password_hash"],
user["honey_password_hashes"],
        )
        if not resolved_login:
            errors["auth_password"] = "incorrect login password"
        if not errors:
            mode, honey_index = resolved_login
entry = fetch_service_entry(
                g.db,
user_id=active_vault["user_id"],
service_name=service_name,
mode=mode,                    vault_key_hex=(
unlock_vault_key(
auth_password,
user["encryption_salt"],

                        "real",
user["real_key_wrap"],
user["honey_key_wraps"],
                        none,
                    )                    if mode
== "real"                    else
unlock_vault_key(
auth_password,
user["encryption_salt"],
```

```
                    "fake",
user["real_key_wrap"],
user["honey_key_wraps"],
honey_index,
                )
            ),          )
if entry is none:
            errors["service_name"] = "no saved password found for this service"
        else:
            result = {"service": service_name, "password": entry, "mode": mode}
return render_template(     "vault.html",     errors=errors,     result=result,
    mode=active_vault["vault_mode"],
  username=active_vault["username"],
  service_names=list_service_names(g.db, active_vault["user_id"]),
  form=request.form,

  )
@app.route("/logout") def logout():
clear_active_vault()    session.clear()
flash("you have been logged out.", "success")
return redirect(url_for("login")) if __name__
== "__main__":
  app.run(host="127.0.0.1", port=5055, debug=false)
```

Code of Crypto Utilities (crypto_utils.py) from

```
cryptography.fernet import fernet from argon2 import
passwordhasher ph = passwordhasher() key =
b'4fyyu1od6chq8pcgroeiqxun56ivnthsfcyyqlyephw='
cipher = fernet(key) def hash_password(password):
```

return ph.hash(password) def verify_password(hash,

password):

try:

return ph.verify(hash, password)

except:

return false def

encrypt(text):

return cipher.encrypt(text.encode()).decode() def

decrypt(text):

return cipher.decrypt(text.encode()).decode()

Code of Database Module (database.py) from __future__ import annotations import secrets

import sqlite3 from datetime import utc, datetime from pathlib import path from security

import build_honey_credentials, decrypt_secret, encrypt_secret, hash_password

latest_schema_version = 2 def get_db(database_path: path) -> sqlite3.connection:

conn = sqlite3.connect(database_path)

conn.row_factory = sqlite3.row

conn.execute("pragma foreign_keys = on")

conn.execute("pragma journal_mode = wal")

conn.execute("pragma synchronous = normal")

conn.execute("pragma busy_timeout = 5000")

return conn def init_db(database_path: path) ->

none:

database_path.parent.mkdir(parents=true, exist_ok=true)

with get_db(database_path) as conn:

apply_migrations(conn)

repair_latest_schema(conn) conn.commit() def

apply_migrations(conn: sqlite3.connection) -> none:

current_version = conn.execute("pragma user_version").fetchone()[0]

if current_version == 0 and table_exists(conn, "users"):

```
        current_version = 1
    set_schema_version(conn, current_version)
    migrations = {
        1: migrate_to_v1,
        2: migrate_to_v2,
    }
    while current_version < latest_schema_version:
        next_version = current_version + 1
migrations[next_version](conn)
set_schema_version(conn, next_version)
current_version = next_version def
migrate_to_v1(conn: sqlite3.connection) -> none:
    conn.executescript(
        """         create table if not exists users (
id integer primary key autoincrement,
username text not null unique,
password_hash text not null,
honey_password_hashes text not null,
encryption_salt text not null,
created_at text not null
        );
        create table if not exists vault_entries (
id integer primary key autoincrement,
user_id integer not null,
service_name text not null,
actual_ciphertext text not null,
honey_ciphertext text not null,
created_at text not null,            updated_at
```

```
text not null,        unique(user_id,
service_name),

            foreign key (user_id) references users(id)
        );
        """
    )
def migrate_to_v2(conn: sqlite3.connection) -> none:
    ensure_column(conn, "users", "real_key_wrap", "text not null default "")
ensure_column(conn, "users", "fake_key_wrap_for_real", "text not null default "")
ensure_column(conn, "users", "honey_key_wraps", "text not null default '[]'") def
repair_latest_schema(conn: sqlite3.connection) -> none:     if table_exists(conn,
"users"):
        ensure_column(conn, "users", "real_key_wrap", "text not null default "")
ensure_column(conn, "users", "fake_key_wrap_for_real", "text not null default "")
ensure_column(conn, "users", "honey_key_wraps", "text not null default '[]'")     if
table_exists(conn, "vault_entries"):
        ensure_column(conn, "vault_entries", "actual_ciphertext", "text not null default "")
ensure_column(conn, "vault_entries", "honey_ciphertext", "text not null default "") def
set_schema_version(conn: sqlite3.connection, version: int) -> none:
    conn.execute(f"pragma user_version = {version}") def
table_exists(conn: sqlite3.connection, table_name: str) -> bool:
    row = conn.execute(
        "select name from sqlite_master where type = 'table' and name = ?",
        (table_name,),     ).fetchone()     return row is not none def column_exists(conn:
sqlite3.connection, table_name: str, column_name: str) -> bool:
    return any(     row["name"] == column_name     for row in
    conn.execute(f"pragma table_info({table_name})").fetchall() )
```

```
def ensure_column(conn: sqlite3.connection, table_name: str, column_name: str, column_sql:
str) -> none:     if not column_exists(conn, table_name, column_name):

        conn.execute(f"alter table {table_name} add column {column_name} {column_sql}") def

create_user(conn: sqlite3.connection, username: str, password: str) -> none:

    password_hash, encryption_salt = hash_password(password)

    honey_hashes, honey_key_wraps, real_key_wrap, fake_key_wrap_for_real, _fake_vault_key
= build_honey_credentials(

password,         encryption_salt,

    )

    now = datetime.now(utc).isoformat()

conn.execute(

        """         insert

into users (

            username, password_hash, honey_password_hashes, encryption_salt, real_key_wrap,
fake_key_wrap_for_real,             honey_key_wraps, created_at

        )         values (? ?, ?, ?, ?,

?, ?, ?)

        """,

        (

            username,

password_hash,

honey_hashes,

encryption_salt,

real_key_wrap,

fake_key_wrap_for_real,

honey_key_wraps,

            now,

        ),
```

```
    )                    conn.commit()    def    get_user_by_username(conn:
sqlite3.connection, username: str):
    return conn.execute(
        """         select             id,
username,
password_hash,
honey_password_hashes,
encryption_salt,
real_key_wrap,
fake_key_wrap_for_real,
honey_key_wraps         from
users         where username = ?
        """,
        (username,),     ).fetchone()
def save_service_secret(     conn:
sqlite3.connection,     user_id:
int,     service_name: str,
actual_secret: str, honey_secret:
str, real_vault_key_hex: str |
none, fake_vault_key_hex: str,
mode: str,
) -> none:     now =
datetime.now(utc).isoformat()  existing
= conn.execute(
        "select actual_ciphertext from vault_entries where user_id = ? and lower(service_name) =
lower(?)",
        (user_id, service_name),
    ).fetchone()     if
mode == "fake":
```

```
        actual_secret = honey_secret         honey_secret =
generate_fake_shadow(actual_secret)         encrypted_honey =
encrypt_secret(fake_vault_key_hex, honey_secret)
        if existing is none:
            encrypted_actual = encrypt_secret(fake_vault_key_hex, actual_secret)
        else:
            encrypted_actual = existing["actual_ciphertext"]
else:         if real_vault_key_hex is none:
            raise valueerror("real vault key is required for real-mode save.")
encrypted_actual = encrypt_secret(real_vault_key_hex, actual_secret)
encrypted_honey = encrypt_secret(fake_vault_key_hex, honey_secret)     conn.execute(
        """
        insert into vault_entries (user_id, service_name, actual_ciphertext, honey_ciphertext,
created_at, updated_at)
        values (?, ?, ?, ?, ?, ?)         on
conflict(user_id, service_name)         do
update set

            actual_ciphertext = excluded.actual_ciphertext,
    honey_ciphertext = excluded.honey_ciphertext,        updated_at
    = excluded.updated_at
        """,
        (user_id, service_name, encrypted_actual, encrypted_honey, now, now),
    )
    conn.commit() def list_service_names(conn: sqlite3.connection,
user_id: int) -> list[str]:
    rows = conn.execute(
        """
```

```
        select service_name
from vault_entries         where
user_id = ?
        order by lower(service_name)
        """,
        (user_id,),
    ).fetchall()     return [row["service_name"]
for row in rows] def fetch_service_entry(
conn: sqlite3.connection,     user_id: int,
service_name: str,     mode: str,
vault_key_hex: str, ) -> str | none:
    row = conn.execute(
        """     select actual_ciphertext,
    honey_ciphertext     from vault_entries
    where user_id = ? and lower(service_name)
    = lower(?)
        """,
        (user_id, service_name),
    ).fetchone()     if row is none:         return none     ciphertext =
row["actual_ciphertext"] if mode == "real" else row["honey_ciphertext"]     return
decrypt_secret(vault_key_hex, ciphertext) def generate_fake_shadow(secret: str) -> str:
if not secret:
        return "decoypass!2026"     return
f"{secret[:2]}vault{len(secret)}!{secrets.randbelow(90) + 10}"
```

Code of Main Application (main.py) from tkinter import * from tkinter import messagebox from database import create_tables, connect from crypto_utils import hash_password, verify_password from vault_core import add_password, get_password, get_fake_password create_tables()

```
current_user_id = none login_mode = "real" root = tk() root.title("offline
password manager") root.geometry("400x300") label(root,
text="username").pack(pady=5) user_entry = entry(root)

user_entry.pack(pady=5) label(root,
text="password").pack(pady=5)
pass_entry = entry(root, show="*")
pass_entry.pack(pady=5) def register():
    username = user_entry.get()
password = pass_entry.get()     if
not username or not password:
        messagebox.showerror("error", "fields cannot be empty")
        return
    honey = password + "123"
conn = connect()     cursor =
conn.cursor()
    try:
        cursor.execute(
            "insert into users (username, password, honey_password) values (?, ?, ?)",
            (username, hash_password(password), hash_password(honey))
        )
        conn.commit()          messagebox.showinfo("success", f"registered!\nhoney
password: {honey}")     except:
        messagebox.showerror("error", "user already exists")
conn.close() def login():
    global current_user_id, login_mode
    username = user_entry.get()
    password = pass_entry.get() conn =
    connect()
```

```
    cursor = conn.cursor()
    cursor.execute("select id, password, honey_password from users where username=?",
(username,))     user =
cursor.fetchone()
conn.close()  if user:
        if verify_password(user[1], password):
            current_user_id = user[0]            login_mode =
"real"            messagebox.showinfo("success", "login
successful")            open_vault()        elif
verify_password(user[2], password):
            current_user_id = user[0]            login_mode = "honey"
messagebox.showwarning("alert", "honey login detected!")
open_vault()        else:
            messagebox.showerror("error", "invalid password")
    else:
        messagebox.showerror("error", "user not found")
def open_vault():     vault = toplevel(root)
vault.title("vault")     vault.geometry("350x320")
label(vault, text="service").pack(pady=5)
service_entry = entry(vault)
service_entry.pack(pady=5)     label(vault,
text="username").pack(pady=5)     username_entry =
entry(vault)     username_entry.pack(pady=5)
label(vault, text="password").pack(pady=5)
password_entry = entry(vault)
password_entry.pack(pady=5)
```

```
    result_label = label(vault, text="")
result_label.pack(pady=10)     def
save():
        service = service_entry.get()         username =
username_entry.get()         password =
password_entry.get()         if not service or not
username or not password:
result_label.config(text="fill all fields")
return
        add_password(current_user_id, service, username, password)
result_label.config(text="saved (not visible)")     def get():
        service = service_entry.get()
if not service:
            result_label.config(text="enter service name")
            return         if
login_mode == "real":
            data = get_password(current_user_id, service)
            if data:
result_label.config(text=f"{data[0]} | {data[1]}")
            else:             result_label.config(text="no
    data found")     else:

            fake_user, fake_pass = get_fake_password(service)
result_label.config(text=f"{fake_user} | {fake_pass}") button(vault,
text="save", command=save).pack(pady=5)     button(vault,
text="get", command=get).pack(pady=5)     button(root,
text="register", command=register).pack(pady=5) button(root,
text="login", command=login).pack(pady=5) root.mainloop()
```

Code of Requriments (requirements.txt)

flask==3.1.0 argon2-cffi==25.1.0

Code of Security Module (security.py) from

__future__ import annotations import hashlib

import hmac import json import secrets from

base64 import b64decode, b64encode from

argon2 import passwordhasher from

argon2.exceptions import verificationerror

password_hasher = passwordhasher(

time_cost=3, memory_cost=65536,

parallelism=2,

hash_len=32,

salt_len=16,

)

def hash_password(password: str) -> tuple[str, str]: salt =

secrets.token_hex(16) return

password_hasher.hash(password), salt def

verify_password(stored_hash: str, candidate: str) -> bool:

try:

return password_hasher.verify(stored_hash, candidate)

except verificationerror:

return false def derive_master_key(password: str,

salt_hex: str) -> str:

key = hashlib.scrypt(

password.encode("utf-8"),

salt=bytes.fromhex(salt_hex),

```
n=2**14,        r=8,        p=1,
dklen=32,
    )
    return key.hex() def
generate_honey_variants(password: str) -> list[str]:
    variants = []     if
len(password) > 2:
        variants.append(password[:-1] + shift_char(password[-1]))
variants.append(password[0] + password[2:3] + password[1:2] + password[3:])
variants.append(password[:1] + password[1:].swapcase())        variants.append(password[:-
2] + shift_char(password[-2]) + password[-1:]) variants.extend(
        [
            password + "1",
            password + "!",        password + "@1",        password[:-1]
if len(password) > 3 else password + "2026",
replace_with_keyboard_neighbor(password),
toggle_first_letter(password),
        ]
    )
    clean_variants = []     for variant in variants:        if variant and
variant != password and variant not in clean_variants:
            clean_variants.append(variant)     return clean_variants[:8] def
build_honey_credentials(password: str, encryption_salt: str) -> tuple[str, str, str, str, str]:
    real_vault_key = secrets.token_hex(32)
fake_vault_key = secrets.token_hex(32)
    real_key_wrap = encrypt_secret(derive_master_key(password, encryption_salt),
real_vault_key)
```

```
    fake_key_wrap_for_real = encrypt_secret(derive_master_key(password, encryption_salt),
fake_vault_key)     honey_hashes = []     honey_key_wraps = []     for variant in
generate_honey_variants(password):
        honey_hashes.append(password_hasher.hash(variant))
        honey_key_wraps.append(encrypt_secret(derive_master_key(variant, encryption_salt),
fake_vault_key))
    return (
json.dumps(honey_hashes),
json.dumps(honey_key_wraps),
real_key_wrap,
fake_key_wrap_for_real,
fake_vault_key,

    )
def verify_password_or_honey(candidate: str, password_hash: str, honey_hashes_json: str) ->
tuple[str, int | none] | none:     if verify_password(password_hash, candidate):
        return ("real", none)   for index, honey_hash in
enumerate(json.loads(honey_hashes_json)):         if
verify_password(honey_hash, candidate):
            return ("fake", index)
return none def
unlock_vault_key(
    candidate: str,
encryption_salt: str,     mode:
str,     real_key_wrap: str,
honey_key_wraps_json: str,
honey_index: int | none,
) -> str:
    derived_key = derive_master_key(candidate, encryption_salt)
if mode == "real":
```

```
        return decrypt_secret(derived_key, real_key_wrap)
honey_key_wraps = json.loads(honey_key_wraps_json)     if
honey_index is none or honey_index >= len(honey_key_wraps):
raise valueerror("unable to resolve honey vault key.")     return
decrypt_secret(derived_key, honey_key_wraps[honey_index]) def
encrypt_secret(master_key_hex: str, plaintext: str) -> str:
    master_key = bytes.fromhex(master_key_hex)
enc_key, mac_key = _split_keys(master_key)     nonce
= secrets.token_bytes(16)

    payload = plaintext.encode("utf-8") keystream = _keystream(enc_key,
nonce, len(payload)) ciphertext = bytes(a ^ b for a, b in zip(payload,
keystream, strict=false))     tag = hmac.new(mac_key, nonce + ciphertext,
hashlib.sha256).digest()     return b64encode(nonce + ciphertext +
tag).decode("ascii") def decrypt_secret(master_key_hex: str, blob: str) ->
str:
    master_key = bytes.fromhex(master_key_hex)     enc_key, mac_key =
_split_keys(master_key)     raw = b64decode(blob.encode("ascii"))     nonce =
raw[:16]     tag = raw[-32:]     ciphertext = raw[16:-32]     expected =
hmac.new(mac_key, nonce + ciphertext, hashlib.sha256).digest()     if not
hmac.compare_digest(expected, tag):
        raise valueerror("encrypted entry failed integrity validation.")
keystream = _keystream(enc_key, nonce, len(ciphertext))     plaintext =
bytes(a ^ b for a, b in zip(ciphertext, keystream, strict=false))     return
plaintext.decode("utf-8") def generate_honey_secret(secret: str) -> str:
    replacements = {
        "a": "@",
        "a": "4",
        "e": "3",
```

```
        "i": "1",
        "o": "0",
        "s": "$",
        "s": "5",
    }
    transformed = "".join(replacements.get(char, char) for char in secret) if
    len(transformed) < 10:
        transformed = f"{transformed}secure!"
if transformed == secret:
        transformed = transformed[:-1] + shift_char(transformed[-1])
    else:
        transformed = transformed[:2] + shift_char(transformed[2:3] or transformed[-1]) +
transformed[3:]     return transformed def shift_char(value: str) -> str:     if not value:
return "x"     if value.isdigit():
        return str((int(value) + 1) % 10)
if value.isalpha():
        start = ord("a") if value.isupper() else ord("a")
return chr(start + ((ord(value) - start + 1) % 26))     return
"!" def replace_with_keyboard_neighbor(password: str) ->
str:
    neighbors = {
        "a": "s",
        "s": "d",
        "d": "f",
        "i": "o",
        "o": "p",
        "1": "2",
        "2": "3",
        "!": "@",
```

```
    }
    if not password:
        return "vault@1234" middle = len(password) // 2     target = password[middle]     return
password[:middle] + neighbors.get(target, shift_char(target)) + password[middle + 1 :] def
toggle_first_letter(password: str) -> str:     if not password:         return "vault@1234"     first =
password[0]     if first.isalpha():
        first = first.swapcase()
else:
        first = shift_char(first)     return first + password[1:]
def _split_keys(master_key: bytes) -> tuple[bytes, bytes]:
    enc_key = hmac.new(master_key, b"enc", hashlib.sha256).digest()
mac_key = hmac.new(master_key, b"mac", hashlib.sha256).digest()
return enc_key, mac_key def _keystream(enc_key: bytes, nonce:
bytes, length: int) -> bytes:
    output = bytearray()     counter = 0     while len(output) < length:         block =
hmac.new(enc_key, nonce + counter.to_bytes(4, "big"), hashlib.sha256).digest()
output.extend(block)         counter += 1     return bytes(output[:length])
```

Code of Home Page (index.html)

```
{% extends "base.html" %}
{% block content %}
<section class="hero">
    <div class="hero-copy">
        <span class="eyebrow">Offline Security Vault</span>
        <h1>Protect every service password in a local encrypted vault.</h1>
        <p>Register once, log in locally, save service credentials into SQLite, and retrieve them
only after password verification. Nothing depends on the internet.</p>
        <div class="hero-actions">
            <a class="button button-primary" href="{{ url_for('register') }}">Create Account</a>
```

```
            <a class="button button-secondary" href="{{ url_for('login') }}">Login</a>
        </div>
    </div>
    <div class="hero-panel">
        <div class="mode-card">
            <div class="mode-row">
                <span>Private storage</span>
                <span>Encrypted locally</span>
            </div>
            <p>Your saved service passwords stay encrypted in local storage until you request one.</p>
        </div>
        <div class="mode-card honey">
            <div class="mode-row">
                <span>Protected access</span>
                <span>Quiet verification</span>
            </div>
            <p>Password verification happens silently in the background to keep retrieval behavior private.</p>
        </div>
        <div class="mode-card">
            <div class="mode-row">
                <span>SQLite + WAL</span>
                <span>Offline only</span>
            </div>
            <p>Your data stays on the device with hardened local database settings and encrypted secret storage.</p>
        </div>
    </div>
</section>
```

```
{% endblock %}
```

Code of Base Template (base.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Offline Password Manager</title>
<link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
</head>
<body>
<div class="page-glow glow-one"></div>
<div class="page-glow glow-two"></div>
<div class="page-grid"></div>
<main class="shell">
{% with messages=get_flashed_messages(with_categories=true) %}
{% if messages %}
<section class="flash-stack">
{% for c,m in messages %}
<div class="flash flash-{{c}}">{{m}}</div>
{% endfor %}
</section>
{% endif %}
{% endwith %}
{% block content %}{% endblock %}
</main>
</body>
</html>
```

Code of Login Page (login.html)

```
{% extends "base.html" %}
{% block content %}
<form method="post">
<input type="hidden" name="csrf_token" value="{{ g.csrf_token }}">
<input type="text" name="username" placeholder="Username">
<input type="password" name="password" placeholder="Password">
<button type="submit">Login</button>
</form>
{% endblock %}
```

Code of Register Page (register.html)

```
{% extends "base.html" %}
{% block content %}
<section class="card auth-card">
  <div class="card-header">
    <span class="eyebrow">Register</span>
    <h2>Create your offline vault</h2>
    <p>Your login password is protected with strong local security before anything is stored.</p>
  </div>
  <form method="post" class="form-grid">
    <input type="hidden" name="csrf_token" value="{{ g.csrf_token }}">
<label>
      <span>Username</span>
      <input type="text" name="username" value="{{ form.get('username', '') }}" placeholder="Enter username" autocomplete="username">
      <small class="error">{{ errors.get('username', '') }}</small>
```

```
        </label>
        <label>
            <span>Master Password</span>
            <input type="password" name="password" placeholder="At least 12 characters with
uppercase, number, symbol" autocomplete="new-password">
            <small class="error">{{ errors.get('password', '') }}</small>
        </label>
        <label>
            <span>Confirm Password</span>
            <input type="password" name="confirm_password" placeholder="Retype master
password" autocomplete="new-password">
            <small class="error">{{ errors.get('confirm_password', '') }}</small>
</label>
        <button class="button button-primary" type="submit">Register Securely</button>
</form>
    <p class="footer-note">Already registered? <a href="{{ url_for('login') }}">Sign in
here</a>.</p> </section>
{% endblock %}
```

Code of Vault Page (vault.html)

```
{% extends "base.html" %}
{% block content %}
<section class="vault-layout">
    <div class="vault-sidebar card">
        <span class="eyebrow">Vault Session</span>
        <h2>{{ username }}</h2>
        <div class="mode-badge mode-neutral">Secure Session</div>
        <p>Your service passwords are stored locally in encrypted form and can be retrieved only
after password verification.</p>
        <div class="service-list">
            <span class="service-list-title">Saved services</span>
```

```
            {% if service_names %}
                {% for service in service_names %}
                    <span class="service-chip">{{ service }}</span>
                {% endfor %}
            {% else %}
                <span class="service-empty">No services saved yet.</span>
            {% endif %}
        </div>
        <a class="button button-secondary" href="{{ url_for('logout') }}">Logout</a>
</div>
    <div class="vault-main">
        <section class="vault-grid">
        <section class="card">
            <div class="card-header">
                <span class="eyebrow">Save Service</span>
                <h2>Store a password securely</h2>
                <p>The service password is encrypted before it reaches SQLite and is never shown
back automatically after saving.</p>
            </div>
            <form method="post" class="vault-form">
                <input type="hidden" name="csrf_token" value="{{ g.csrf_token }}">
                <label>
                    <span>Service Name</span>
                    <input type="text" name="service_name" value="{{ form.get('service_name', '') }}"
placeholder="Example: Gmail, GitHub, Banking">
                    <small class="error">{{ errors.get('service_name', '') }}</small>
                </label>
                <label>
                    <span>Service Password</span>
```

```
                <input type="password" name="service_password" placeholder="Needed only when
saving">
                <small class="error">{{ errors.get('service_password', '') }}</small>
</label>
            <button class="button button-primary" type="submit" name="action"
value="save">Save Password</button>
        </form>
    </section>
    <section class="card">
        <div class="card-header">
            <span class="eyebrow">Get Service</span>
            <h2>Retrieve a password carefully</h2>
            <p>Enter the service name and your login password to retrieve the saved password for
that service.</p>
        </div>
        <form method="post" class="vault-form">
            <input type="hidden" name="csrf_token" value="{{ g.csrf_token }}">
            <label>
                <span>Service Name</span>
                <input type="text" name="service_name" value="{{ form.get('service_name', '') }}"
placeholder="Enter the saved service name">
                <small class="error">{{ errors.get('service_name', '') }}</small>
            </label>
            <label>
                <span>Login Password For Retrieval</span>
                <input type="password" name="auth_password" placeholder="Needed only when
getting a password">
             <small class="error">{{ errors.get('auth_password', '') }}</small>
</label>
            <button class="button button-secondary" type="submit" name="action"
value="get">Get Password</button>
```

```
            </form>
        </section>
        </section>
        <section class="card result-card">
            <div class="card-header">
                <span class="eyebrow">Result</span>
                <h2>Retrieved service password</h2>
            </div>
            {% if result %}
                <div class="result-pill mode-neutral">Password Ready</div>
                <div class="result-panel">
                    <h3>{{ result.service }}</h3>
                    <input class="result-input" type="text" value="{{ result.password }}" readonly>
                </div>
            {% else %}
                <p class="result-empty">Nothing retrieved yet. Use the Get action with the service
name and your login password.</p>
            {% endif %}
        </section>
    </div>
</section>
{% endblock %}
```

Code of Stylesheet (styles.css)

```
:root {
    --bg: #07131f;
     --panel: rgba(9, 24, 38, 0.78);
    --panel-strong: rgba(8, 20, 31, 0.92);
    --panel-border: rgba(112, 198, 204, 0.18);
```

```
    --text: #eff8ff;
    --muted: #adc3d3;
    --primary: #7ce3c7;
    --primary-strong: #2ec39a;
    --secondary: #ffc474;
    --danger: #ff98a8;
    --accent: #7fb7ff;
    --shadow: 0 24px 70px rgba(0, 0, 0, 0.35);
} * {     box-sizing:
border-box;
} body {     margin: 0;     min-height: 100vh;    font-family: "Bahnschrift", "Segoe UI
Variable Text", "Trebuchet MS", sans-serif;     color: var(--text);     background:
        radial-gradient(circle at top left, rgba(38, 198, 161, 0.22), transparent 28%),         radial-
gradient(circle at top right, rgba(255, 196, 116, 0.18), transparent 24%),         linear-
gradient(145deg, #07121d, #0d2333 54%, #0b1724);
} .page-grid {
position: fixed;
inset: 0;
background-image:
        linear-gradient(rgba(255, 255, 255, 0.04) 1px, transparent 1px),
linear-gradient(90deg, rgba(255, 255, 255, 0.04) 1px, transparent 1px);
background-size: 36px 36px;     mask-image: linear-gradient(180deg, rgba(0,
0, 0, 0.36), transparent 84%);     pointer-events: none;
}
.page-glow {
position: fixed;
width: 18rem;
height: 18rem;
border-radius: 50%;
```

```
filter: blur(70px);
z-index: 0;     opacity:
0.45; }
.glow-one {
top: 4rem;
left: 2rem;
background:
rgba(112, 225,
200, 0.35);

}
.glow-two {     right: 3rem;     bottom:
2rem;     background: rgba(255, 184,
107, 0.3);
} .shell {     position: relative;     z-index:
1;     width: min(1120px, calc(100% -
2rem));     margin: 0 auto;     padding:
2rem 0 3rem;
} .flash-stack {
display: grid;     gap:
0.8rem;     margin-
bottom: 1rem;
}
.flash {     padding: 1rem 1.2rem;     border-
radius: 18px;     backdrop-filter: blur(10px);
border: 1px solid rgba(255, 255, 255, 0.12);
} .flash-success {     background:
rgba(38, 198, 161, 0.15);
}
```

```
.flash-warning {     background:
rgba(255, 184, 107, 0.16);
} .flash-error {     background: rgba(255,
141, 152, 0.16);
}
.hero, .vault-layout {     display: grid;
gap: 1.4rem; } .hero {     grid-template-
columns: 1.15fr 0.85fr;     align-items:
center;     min-height: calc(100vh -
6rem);
}
.hero-copy h1,
.card-header h2,
.result-panel h3, .vault-sidebar h2 {     margin: 0;
font-family: "Georgia", "Palatino Linotype", serif;
} .hero-copy h1 {     font-size:
clamp(2.5rem, 5vw, 4.5rem);     line-
height: 1.08;     max-width: 11ch;
}
.hero-copy p,
.card-header p,
.vault-sidebar p,
.footer-note, .result-
empty {     color:
var(--muted);     line-
height: 1.7; }
.hero-actions, .actions
{     display: flex;
gap: 1rem;     flex-
```

```
wrap: wrap;
margin-top: 1.5rem;
}
.hero-panel,
.card,
.mode-card {     border: 1px solid var(--
panel-border);     background: var(--
panel);     box-shadow: var(--shadow);
backdrop-filter: blur(18px);
}
.hero-panel, .card {
border-radius: 28px; }
.hero-panel {
    padding: 1.2rem;
    display: grid;
    gap: 1rem;
} mode-card {     padding: 1.2rem;     border-radius: 22px;     background: linear-
gradient(180deg, rgba(255, 255, 255, 0.04), rgba(6, 18, 29, 0.72)); }
.mode-card.honey {       background: linear-gradient(135deg, rgba(255, 184, 107, 0.12),
rgba(10, 18, 32, 0.85)); }
.mode-row {
    display: flex;     justify-
content: space-between;     gap:
1rem;     font-weight: 700;
} .card {
padding: 1.5rem;
}
```

```
.auth-card {     width:
min(560px, 100%);
margin: 3rem auto;
}
.card-header {     margin-
bottom: 1rem;
}
.eyebrow {
   display: inline-flex;     margin-bottom:
0.6rem;     padding: 0.3rem 0.7rem;
border-radius: 999px;     background:
rgba(112, 225, 200, 0.14);     color: var(--
primary);     font-size: 0.85rem;     letter-
spacing: 0.08em;     text-transform:
uppercase;
}
.form-grid, .vault-form {     display: grid;
gap: 1rem; } label span {     display: block;
margin-bottom: 0.45rem;     font-weight:
700; } input {     width: 100%;     padding:
0.95rem 1rem;     border-radius: 16px;
border: 1px solid rgba(182, 195, 218, 0.2);
background: rgba(255, 255, 255, 0.06);
color: var(--text);
   font: inherit; outline: none; transition: border-color 0.2s ease, transform 0.2s
   ease, background 0.2s ease;
} input:focus {     border-color: rgba(112,
225, 200, 0.75);     transform: translateY(-
```

```
1px);     background: rgba(255, 255, 255,
0.09);
} .error {     min-
height: 1rem;
display: block;
margin-top: 0.4rem;
color: var(--danger);
} .button {     display:
inline-flex;     justify-
content: center;     align-
items: center;     gap:
0.5rem;     padding: 0.95rem
1.2rem;     border-radius:
16px;     border: none;
text-decoration: none;
font-weight: 800;
    font-family: "Bahnschrift SemiBold", "Segoe UI Variable Display", "Trebuchet MS",
sansserif;     cursor: pointer;     transition: transform 0.2s ease, box-shadow 0.2s ease;
}
.button:hover {     transform:
translateY(-2px);
}
.button-primary {     background: linear-gradient(135deg, var(--primary), var(-
-primary-strong));     color: #052218;
}
.button-secondary {   background: rgba(255,
255, 255, 0.08);     color: var(--text);
border: 1px solid rgba(255, 255, 255, 0.12);
}
```

```
.vault-layout {     grid-template-
columns: 320px 1fr;     align-items:
start;
}
.vault-main {
display: grid;
gap: 1.2rem; }
.vault-grid {     display: grid;     grid-template-
columns: repeat(2, minmax(0, 1fr));     gap: 1.2rem;
}
.mode-badge,
.result-pill {
    display: inline-flex; align-
items: center; width: fit-
content;     padding:
0.45rem 0.8rem;     border-
radius: 999px;     font-
weight: 800;     margin:
0.75rem 0 1rem;
}
.mode-real {     background: rgba(38,
198, 161, 0.16);     color: var(--
primary);
}
.mode-fake {     background: rgba(255,
184, 107, 0.16);     color: var(--
secondary);
}
```

```
.mode-neutral {     background: rgba(127,
183, 255, 0.14);     color: var(--accent);
}
.result-panel {     padding: 1.2rem;     border-radius: 20px;     background: linear-
gradient(180deg, rgba(255, 255, 255, 0.06), rgba(9, 24, 38, 0.9));     border: 1px solid
rgba(255, 255, 255, 0.08);
}
.result-panel p {
margin: 0.4rem 0 0;
font-size: 1.05rem;
word-break: break-
word;
} .result-input {     margin-top: 0.7rem;
background: rgba(4, 16, 26, 0.92);
border-color: rgba(127, 183, 255, 0.28);
font-weight: 700;     letter-spacing:
0.03em;
} .service-list {     display:
flex;     flex-wrap: wrap;
gap: 0.55rem;     margin:
1.2rem 0 1.4rem; }
.service-list-title {
width: 100%;     color:
var(--muted);     font-
size: 0.95rem; }
.service-chip,
.service-empty {     display: inline-flex;
align-items: center;     padding: 0.45rem
0.8rem;     border-radius: 999px;
```

```
background: rgba(255, 255, 255, 0.06);
border: 1px solid rgba(255, 255, 255,
0.08);
}
.service-empty {
color: var(--muted);
}
.footer-note a {     color:
var(--primary);
}
@media (max-width: 860px) {
     .hero,
    .vault-layout,     .vault-grid {
grid-template-columns: 1fr;
    }
    .hero {         min-
height: auto;
padding-top: 2rem;
    }
    .shell {         width: min(100% -
1rem, 1120px);
    }
}
```

4. RESULTS

The developed system successfully performs the following operations:

- user registration with validation
- secure login with master password
- local storage of encrypted service passwords

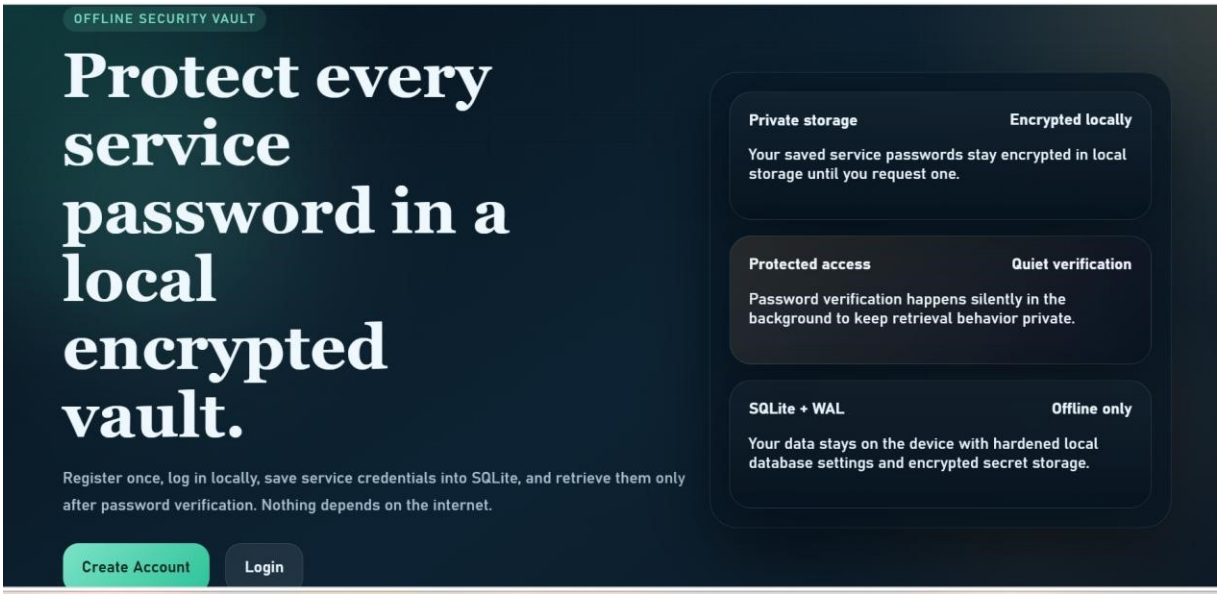
- retrieval of real passwords with correct master password
- retrieval of honey passwords with similar/honey password

- preservation of real service password even after honey-mode interaction
- secure logout and session clearing

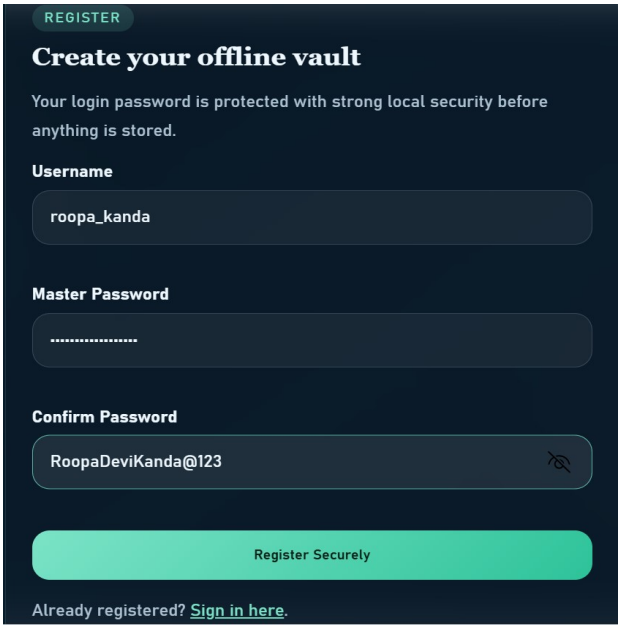
The system runs fully offline and provides a clean interface for password storage and retrieval.

5. OUTPUT SCREENS

- **Fig 1. Home Page**

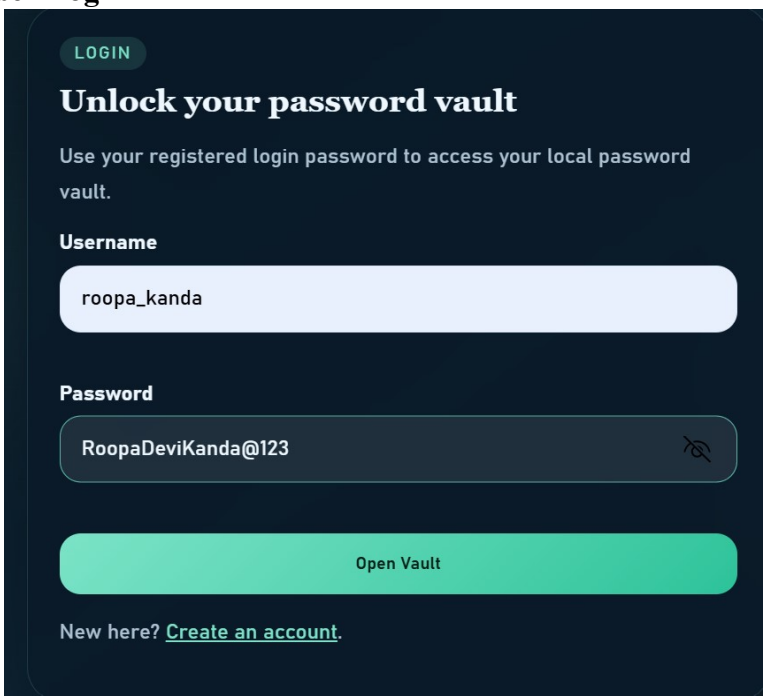


- **Fig 2. User Registration**



•

Fig 3. User Login



• **Fig 4. Vault Dashboard**

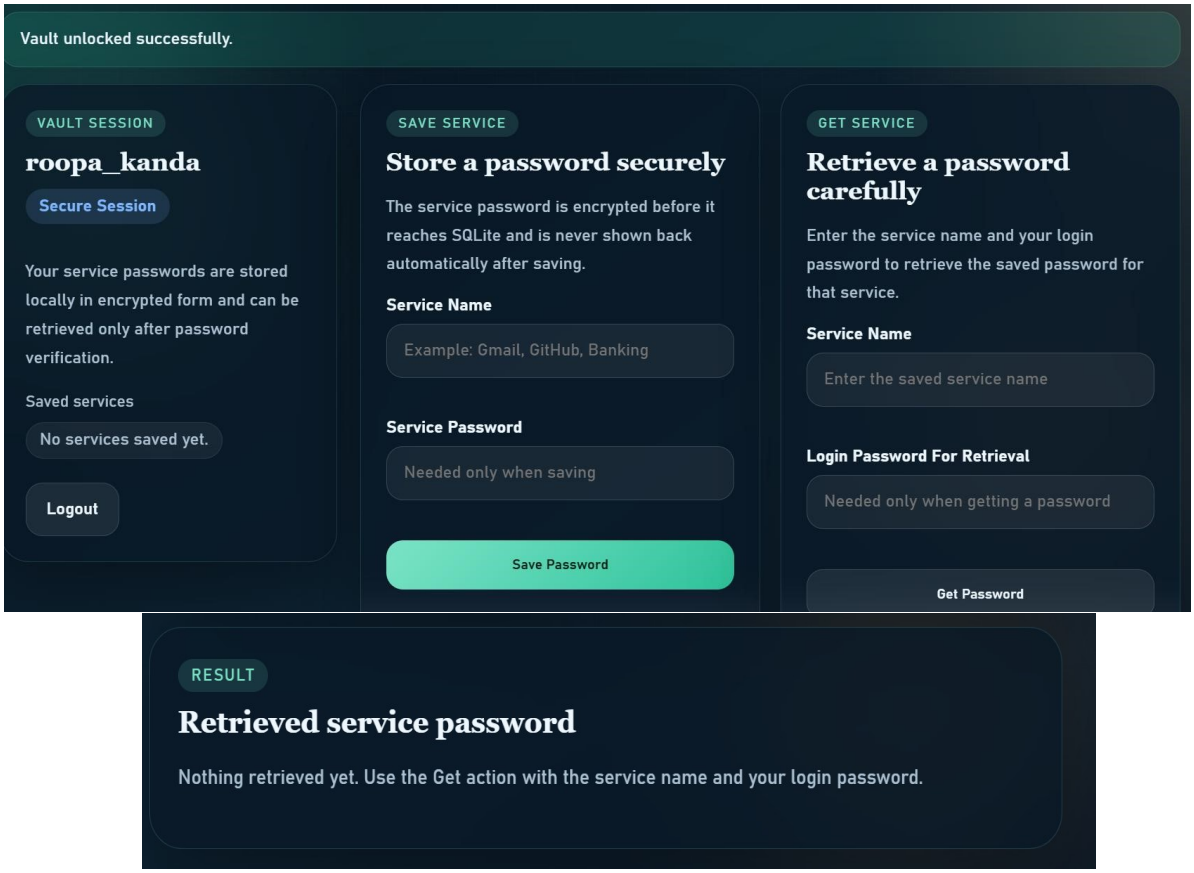
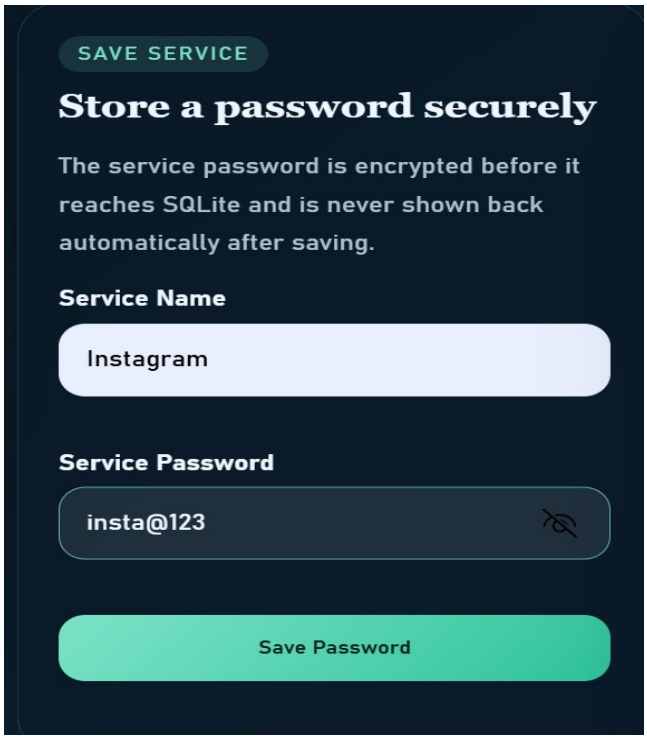


Fig 5. Saving Service Password

•



- **Fig 6. Real Password Retrieval**

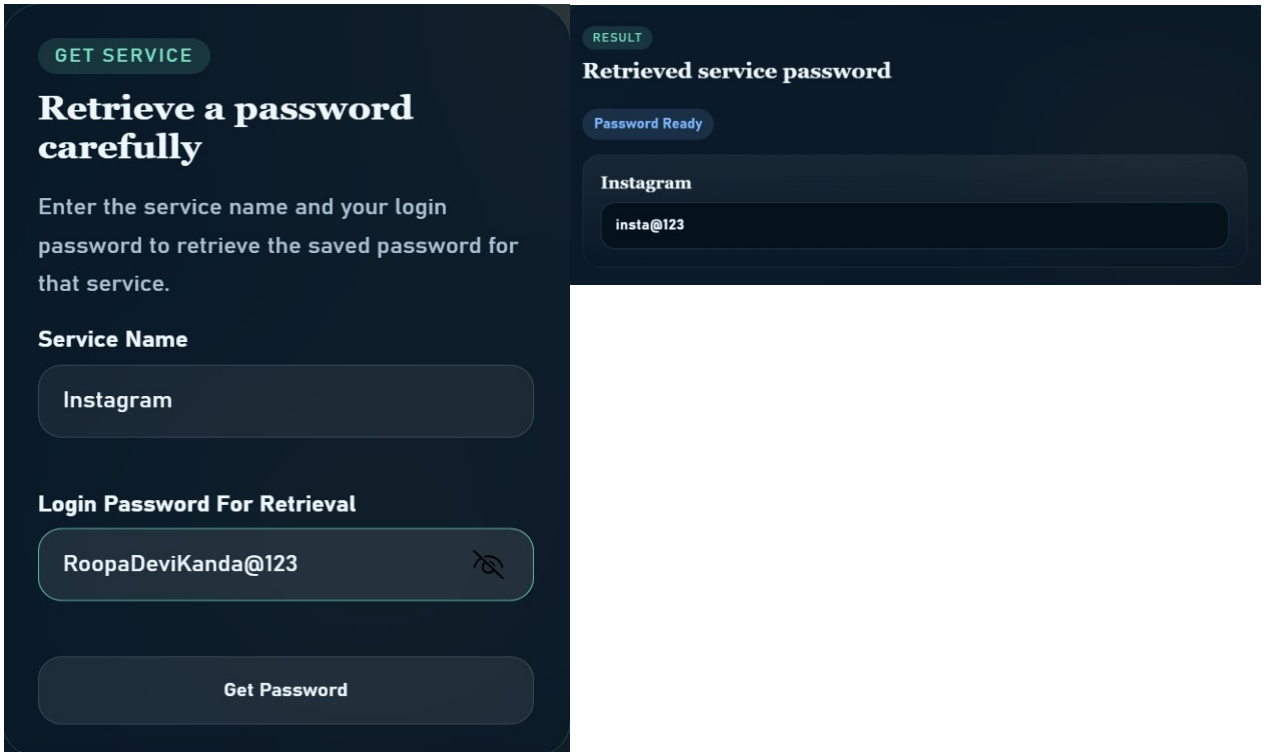
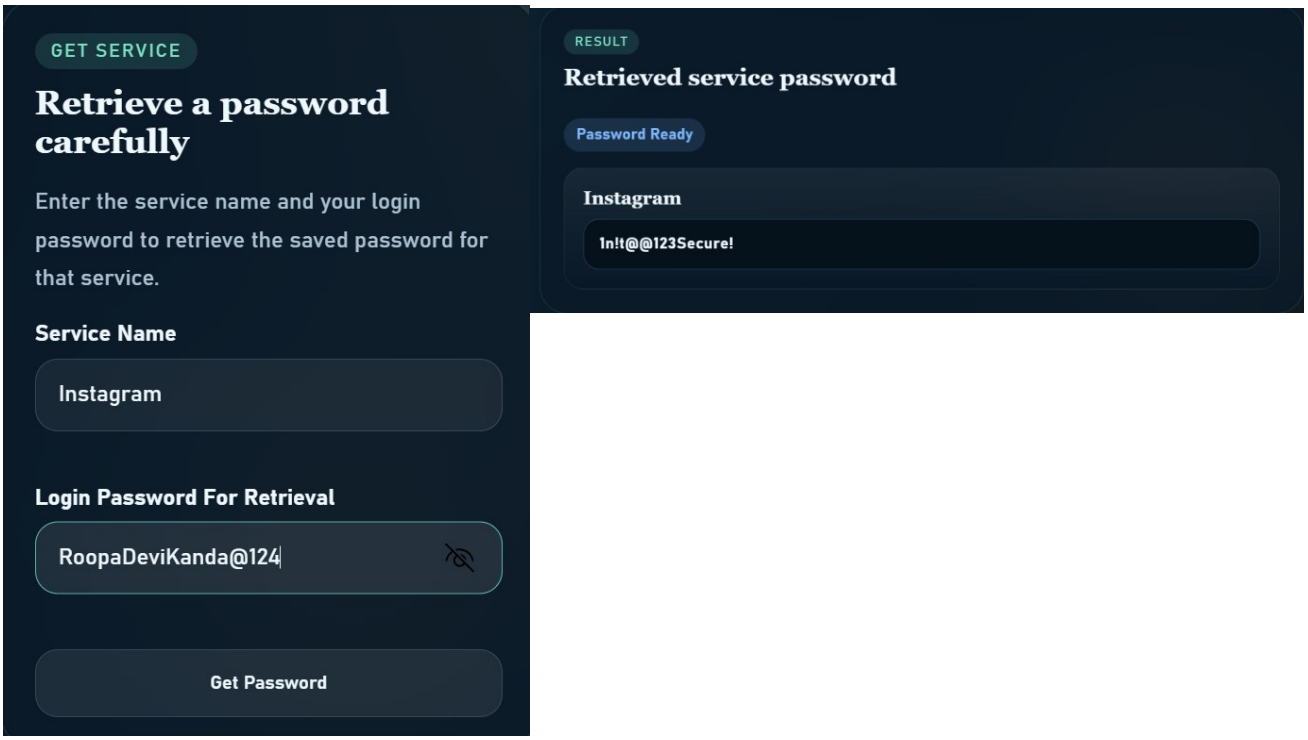
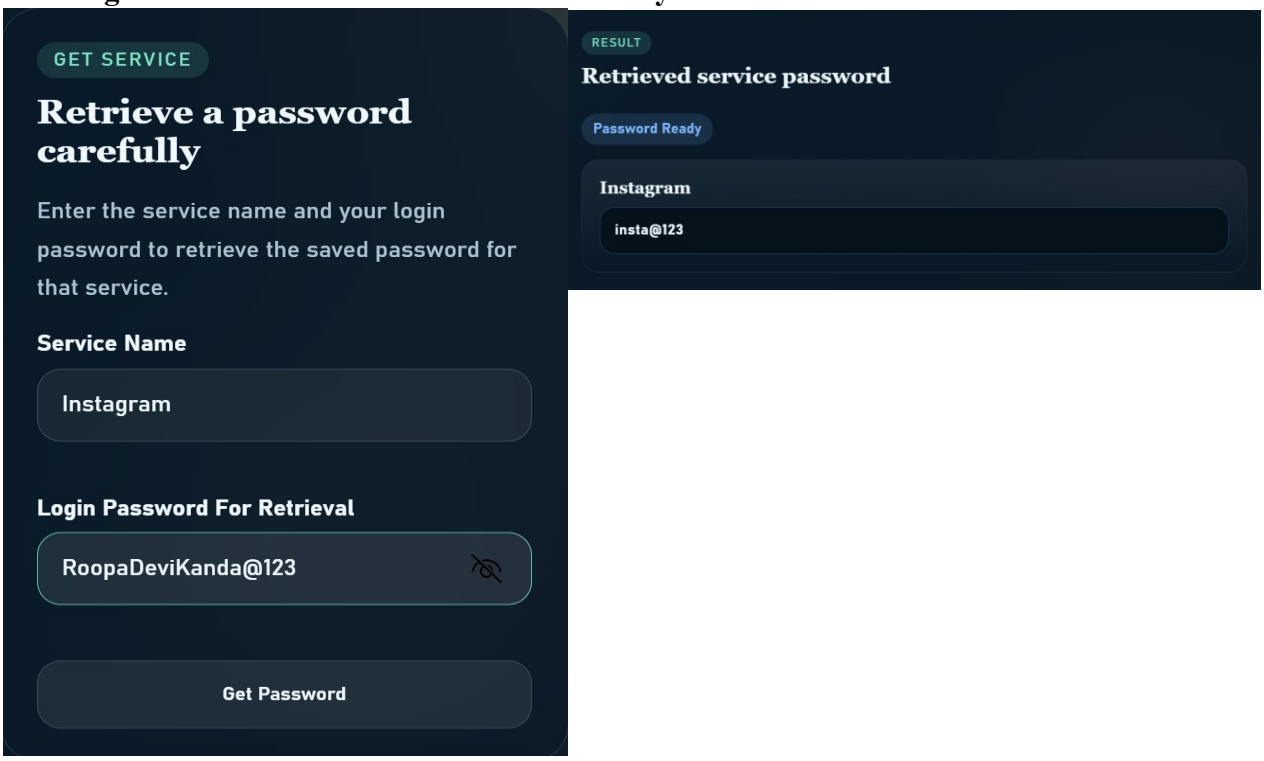


Fig 7. Honey Password Retrieval

•



• **Fig 8. Real Password Retrieval After Honey Mode**



6. CONCLUSION

The **Offline Password Manager** provides a secure and practical solution for managing passwords locally. It enables users to store and retrieve service passwords safely without relying on internet-based services. By combining encryption, password hashing, local database storage, and honey password techniques, the system enhances security against unauthorized access and deceptive login attempts.

The project shows that an offline password manager can be lightweight yet effective while incorporating advanced protection mechanisms. It is especially beneficial for users who prefer privacy, local control, and reduced dependence on cloud-based storage.

The use of honey password techniques improves security by misleading potential attackers with believable fake data when incorrect credentials are entered. This adds an extra layer of defence beyond traditional authentication systems.

The modular design ensures maintainability and scalability, supporting future enhancements such as multi-user support, biometric authentication, and hardware security integration. Secure coding practices further help protect user data from common vulnerabilities.

Overall, the project emphasizes the importance of combining usability with strong security principles. It demonstrates that a locally deployed application can implement modern security strategies effectively and serves as a foundation for more advanced password management solutions.

7. FUTURE SCOPE

The future enhancements of the project may include:

- Implement **Multi-Factor Authentication (MFA)** using OTP, fingerprint, or face recognition
- Add **multi-user support** with separate secure vaults for different users
- Enable **cloud synchronization with end-to-end encryption** for access across devices
- Develop a more **interactive and responsive user interface**
- Introduce **advanced honey encryption techniques** for stronger security
- Add **intrusion detection and alert system** for suspicious login attempts
- Implement **automatic strong password generator**
- Provide **password strength checker** and security suggestions
- Add **password expiry reminders** and notifications
- Include **backup and recovery options** for data safety
- Enable **browser integration** for autofill functionality
- Convert the system into a **desktop or mobile application** for better accessibility

8. REFERENCES

1. Flask Official Documentation, https://flask.palletsprojects.com/
2. Python Official Documentation, https://docs.python.org/3/
3. SQLite Documentation, https://www.sqlite.org/docs.html
4. Argon2 Password Hashing Documentation, https://argon2-cffi.readthedocs.io/
5. NIST Digital Identity Guidelines for Password Security
6. Research papers and articles on honey passwords and deception-based authentication